THEMAJORD'HOME

Guide technique *pour le lancement*

Site, panneau d'administration, réservations (Beds24 + Stripe), Google Ads et mise en ligne — ce qui est fait, ce qui est préparé et ce qu'il reste à faire.

Destinataire : ingénieur chargé de la mise en ligne et des intégrations
Date : 2 novembre 2026
Dépôt : github.com/Jesunilson-Mateus/Project-Themjordhomewebsite (branche main)
Documents liés dans le dépôt : docs/INTEGRACAO-BEDS24-STRIPE.md · README.md (en portugais)

Sommaire

1. Résumé et état actuel
2. Le code et la structure du projet
3. Panneau d'administration
4. Réservations : Beds24 + Stripe
5. Publier, mettre à jour et sauvegardes
6. SEO technique (avant la publicité)
7. Google Ads et mesure
8. Points en suspens et points d'attention
9. Checklist de lancement

Comment lire ce document. Les sections 1 à 3 décrivent l'existant. La section 4 concerne le branchement des comptes Beds24 et Stripe. Les sections 5 à 7 couvrent la mise en ligne, le SEO et Google Ads. La section 8 liste ce qui **n'est pas encore résolu** — dont des points juridiques à traiter avant d'accepter des paiements.

1. Résumé et état actuel

Le site TheMajord'Home est un site de location saisonnière à Porto (60 appartements) avec des pages de services pour les propriétaires (gestion, conciergerie, ménage, maintenance). C'est un site statique en HTML/CSS/JS, avec un panneau d'administration en PHP pour que l'équipe modifie les contenus, et une API de réservation en PHP prête pour Beds24 et Stripe, actuellement **désactivée** (mode démonstration).

Composant	État	Notes
Site public (PT / EN / FR)	**Prêt**	14 pages, thème clair/sombre, adapté au mobile (vérifié de 320 à 1280 px).
Recherche dans la collection	**Prêt**	Nom, quartier et nombre de voyageurs (1–10), sur l'accueil et apartments.html.
Panneau d'administration	**Prêt**	Appartements (actif/inactif), photos, équipements, galeries des services, avis, utilisateurs, mode maintenance. Nécessite PHP.
Pages 404 et maintenance	**Prêt**	404.html et manutencao.html (PT/EN/FR), branchées par le .htaccess racine (Apache). Testées avec Apache.
Données (data/*.json)	**Prêt**	Source de vérité. Le panneau et scripts/sync-data.js régénèrent les fichiers .js.
Réservations — Beds24	**Préparé, désactivé**	API dans api/. Reste : identifiants, table des chambres, valider 4 réponses de l'API.
Paiements — Stripe	**Préparé, désactivé**	Checkout + webhook prêts et testés avec un service simulé. Reste : les clés.
E-mails de réservation	**Basique**	Utilise mail() de PHP. À remplacer par SMTP / service transactionnel.
Google Ads / Analytics	**À faire**	Le bandeau cookies expose déjà le consentement (window.TMHConsent).
SEO technique	**À faire**	Pas de meta description, favicon, robots.txt, sitemap.
Textes juridiques	**À refaire**	Les mentions légales sont un modèle français. Manquent : confidentialité, cookies, conditions de réservation.
Hébergement et domaine	**À décider**	Hébergement avec PHP 8.1+ obligatoire. Netlify ne convient pas.

Hébergement : le site nécessite PHP (panneau, API de réservation, page 404 et mode maintenance). Les hébergements de fichiers statiques uniquement (p. ex. Netlify, GitHub Pages) ne conviennent pas.

2. Le code et la structure du projet

Où se trouve le code

- Dépôt : `github.com/Jesunilson-Mateus/Project-Themjordhomewebsite` — branche `main` (public).
- La copie de travail est celle du dépôt. Il existe une ancienne copie dans Downloads\majordhome-project (septembre) qu'il **ne faut pas utiliser**, sauf pour récupérer le fichier admin/LEIA-ME.txt (voir section 3).
- Pas d'étape de compilation (build) : les fichiers sont publiés tels quels.

Dossiers principaux

Dossier / fichier	Contenu
`*.html`	Pages : index, apartments, property (modèle de chaque appartement), search-results, proprietarios, servico-*, reserva, legal-notice ; review.php (avis).
`css/style.css`	Tout le style du site (variables de thème dans :root).
`js/`	main.js (accueil, collection, recherche), property.js, booking-*.js (réservations), i18n.js (traductions), cookie-consent.js, map.js.
`data/`	Source de vérité : properties.json, translations.json, amenity-*.json, nearby-labels.json, service-galleries.json, beds24-rooms.json, reviews.json. Protégé par .htaccess.
`img/`	Photos de chaque appartement (img/<slug>/) et des services (img/servicos/).
`admin/`	Panneau d'administration (PHP).
`api/`	API de réservation (PHP) : Beds24 + Stripe.
`scripts/sync-data.js`	Régénère les .js et les galeries des services à partir des JSON.
`docs/`	Ce guide et le détail technique de l'intégration Beds24/Stripe.

Règle des données

Les fichiers de `data/` sont la source de vérité. Le site lit des copies intégrées dans `js/properties-data.js`, `js/i18n.js` et dans les galeries des pages `servico-*.html`. Le panneau met tout à jour automatiquement ; si l'on modifie les JSON à la main, il faut exécuter :

```
node scripts/sync-data.js          # met à jour les fichiers générés
node scripts/sync-data.js --check  # vérifie seulement (utile en CI)
```

Le script refuse de s'exécuter si la synchronisation devait supprimer des entrées qui n'existent que dans les .js.

Lancer en local

```
php -S 127.0.0.1:8080 -t .      # à la racine du projet ; ouvrir http://127.0.0.1:8080
```

Ouvrir les fichiers par double-clic affiche aussi le site, mais le panneau et l'API ne fonctionnent qu'avec PHP.

3. Panneau d'administration

Accessible sur `/admin/`. Permet à l'équipe, sans toucher au code :

- de modifier chaque appartement : nom, capacité, descriptions (PT/EN/FR), équipements, règlement intérieur ;
- de gérer les photos (envoi, ordre, photo de couverture, suppression) — redimensionnées à 1800 px ;
- de modifier les galeries « En images » des 4 pages de services (onglet Serviços) ;
- d'approuver les avis des voyageurs et de générer des liens d'invitation (review.php) ;
- d'activer / désactiver des appartements : un appartement inactif disparaît du site et ne peut pas être réservé (données conservées) ;
- d'activer / désactiver le **mode maintenance** (onglet Manutenção) : les visiteurs voient alors manutencao.html ;
- de gérer les utilisateurs du panneau.

Identifiants

> Le fichier `admin/LEIA-ME.txt` contient les instructions et le mot de passe du premier compte. Il est hors de git (c'est voulu) et **n'existe que dans l'ancienne copie de Downloads**. Il doit être transmis à l'ingénieur par un canal sécurisé (pas par simple e-mail). Le fichier `admin/users.json` est créé sur le serveur ; le mot de passe initial doit être changé à la première connexion.

Exigences côté serveur

- PHP 8.1+ avec les extensions **gd** (photos), **curl** (API), **mbstring**.
- PHP doit pouvoir écrire dans : `data/`, `data/backups/`, `data/reservations/`, `img/`, `admin/users.json`, `js/properties-data.js`, `js/i18n.js`, `js/reviews-data.js` et dans les 4 pages `servico-*.html`.
- Sauvegardes automatiques avant chaque enregistrement : tous les data/*.json et les comptes du panneau, dans `data/backups/AAAA-MM-DD_HHMMSS/` (les 60 dernières) — voir section 5.
- Protections existantes : `admin/.htaccess` (users.json et .txt/.md), `data/.htaccess` (dossier entier), `api/.htaccess` (config et bibliothèques). Sous **nginx**, ces règles doivent être recréées.
- Recommandé : HTTPS uniquement ; envisager de restreindre /admin/ par IP ou avec une authentification supplémentaire.

4. Réservations : Beds24 + Stripe

L'intégration est développée et testée contre des services simulés, mais désactivée. Le détail complet (contrat de l'API, codes d'erreur, exemples JSON, en portugais) se trouve dans `docs/INTEGRACAO-BEDS24-STRIPE.md`.

Déroulement d'une réservation

1. Le calendrier demande au serveur les nuits occupées, que celui-ci lit dans Beds24.
2. Le récapitulatif affiche le prix calculé par Beds24 — **toujours côté serveur**, jamais transmis par le navigateur.
3. À la confirmation, le serveur crée la réservation dans Beds24 au statut request (les dates sont bloquées) et ouvre une page de paiement Stripe Checkout (valable 30 min).
4. Stripe appelle le webhook ; la réservation passe à confirmed dans Beds24 et des e-mails sont envoyés à l'équipe et au voyageur.
5. Si le client ne paie pas à temps, le webhook d'expiration annule la réservation et libère les dates.
6. Le client revient sur `reserva.html`, qui affiche l'état (en attente / confirmée / annulée).

Fichiers

Fichier	Rôle
`js/booking-config.js`	Interrupteur du site : mode 'mock' (actuel) ou 'live'.
`js/booking-api.js`	Seule couche du navigateur qui communique avec l'API.
`api/config.example.php`	Modèle → copier en api/config.php sur le serveur (hors git).
`api/beds24.php`	Client Beds24 v2 : authentification prête ; 4 points marqués VERIFICAR.
`api/stripe.php`	Checkout Session et vérification de la signature du webhook (prêt).
`api/*.php`	Endpoints : availability, search, quote, reservations, stripe-webhook.
`data/beds24-rooms.json`	Table appartement → roomId Beds24 (60 entrées à remplir).

Étapes — Beds24

1. Beds24 → Settings → Apps & Integrations → API : créer un invite code (lecture de l'inventaire et des propriétés ; lecture/écriture des réservations). Il expire en 24 h.
2. Échanger le code contre un refresh token (une seule fois) via `/authentication/setup` ; l'enregistrer dans `beds24.refresh_token`. Les tokens de 24 h sont générés et mis en cache automatiquement. Le refresh token expire après 30 jours sans utilisation.
3. Remplir `data/beds24-rooms.json` avec le roomId de chacun des 60 appartements.
4. Vérifier dans le Swagger (beds24.com/api/v2), avec le compte réel, le format des réponses de availability, offers et POST /bookings, ainsi que le champ de notes internes utilisé lors de la confirmation.
5. Recommandé : enregistrer le paiement Stripe dans la réservation (invoiceItems) pour que le solde apparaisse payé.

Étapes — Stripe

1. Clé secrète (`sk_test_…` pour les tests, puis `sk_live_…`) dans `stripe.secret_key`.
2. Webhook dans le tableau de bord Stripe : `https://<domaine>/api/stripe-webhook.php` avec les événements `checkout.session.completed` et `checkout.session.expired` ; copier le secret whsec_… dans `stripe.webhook_secret`.
3. `charge_percent` : 100 encaisse tout ; 30 encaisse un acompte de 30 %. `enabled: false` désactive le paiement en ligne (les réservations restent des demandes et l'équipe confirme par e-mail).

4. Tester avec les clés de test et la carte 4242 4242 4242 4242 avant de passer aux clés live.

Activer

1. `api/config.php` : mode = 'live' et site_url correct.
2. `js/booking-config.js` : mode: 'live'.
3. Mettre à jour le texte `property.finePrint` dans js/i18n.js (il indique encore que les prix en temps réel « arrivent bientôt »).

À décider avant la production

- **Recherche :** api/search.php fait 1 requête Beds24 par appartement (60 par recherche) ; Beds24 limite à ~100 crédits par 5 min et par compte → regrouper les requêtes ou utiliser un cache (~10 min).
- **E-mails :** remplacer mail() par SMTP ou un service transactionnel ; configurer SPF, DKIM et DMARC sur le domaine.
- **Anti-spam** sur le POST de réservation (chaque demande crée une réservation en attente dans Beds24).
- **Remboursements et annulations :** aucun remboursement automatique ; définir une politique (voir section 8).
- **Alternative plus simple :** utiliser le moteur de réservation de Beds24, qui intègre Stripe nativement — on perd l'assistant du site, mais l'API n'est plus nécessaire.

5. Publier, mettre à jour et sauvegardes

Hébergement

- Hébergement avec **PHP 8.1+** (gd, curl, mbstring) — p. ex. cPanel/LAMP, comme celui du site actuel themajordhome.com.
- HTTPS obligatoire (Let's Encrypt ou équivalent), avec redirection http → https et choix www / sans www.
- Apache : les .htaccess existants fonctionnent. nginx : recréer les règles de blocage (data/, admin/users.json, api/lib|beds24|stripe|config.php, fichiers .txt/.md de admin) et celles du .htaccess racine (ci-dessous).
- Le site doit être à la **racine du domaine** (404.html et manutencao.html utilisent <base href="/">).

Le .htaccess racine

- `ErrorDocument 404 /404.html` : page pour les adresses inexistantes (avec X-Robots-Tag noindex).
- **Mode maintenance :** si `data/maintenance.flag` existe (créé par le panneau), tout le site public répond avec `manutencao.html` et le code 503 + Retry-After. Restent accessibles : /admin/, `api/stripe-webhook.php`, css/, js/, img/. Qui active la maintenance reçoit le cookie `tmh_preview=1` et continue de voir le site (commodité, pas une protection).
- Bloque `.git/`, les autres fichiers cachés (sauf .well-known) et `_arquivo/` — important si le site est publié avec git clone.
- Nécessite mod_rewrite et mod_headers. Sous nginx : error_page 404/503, un if (-f …/data/maintenance.flag) avec les mêmes exceptions, et location ~ /\. { return 404; }.

Étapes

1. Faire pointer le domaine (DNS) vers l'hébergement et activer HTTPS.
2. Publier le contenu du dépôt (p. ex. `git clone` ou copie SFTP). **Ne pas publier** `.claude/`, `_arquivo/` (hors git) ni le fichier isolé ADMIN_PANEL_PREVIEW.html.
3. Créer sur le serveur (hors git) : `api/config.php` et le compte du panneau (admin/LEIA-ME.txt).
4. Donner à PHP les droits d'écriture sur les dossiers et fichiers listés en section 3.
5. Désactiver l'affichage des erreurs PHP en production (display_errors = Off) et conserver les logs.

6. Tester : pages, langues, panneau (/admin/), recherche, puis — en mode live avec les clés de test — une réservation complète jusqu'au webhook.
7. Activer les clés live de Stripe et passer les deux interrupteurs en 'live'.

Mettre à jour le site

> **Une fois le site en ligne, les données de référence sont celles du serveur** — l'équipe les modifie dans le panneau. Lors d'une mise à jour, ces fichiers **ne doivent pas être remplacés** par les versions du dépôt : `data/`, `img/`, `admin/users.json`, `api/config.php`, `js/properties-data.js`, `js/i18n.js`, `js/reviews-data.js` et `servico-*.html`. S'il faut en modifier un (p. ex. les textes de l'interface dans js/i18n.js), rapatrier d'abord la version du serveur dans le dépôt.

1. Dans le panneau, **activer le mode maintenance**.
2. **Faire une copie** du serveur : `data/` (y compris data/backups/ et data/reservations/), `img/`, `admin/users.json`, `api/config.php`.
3. Publier les nouveaux fichiers de code, sans remplacer ceux de la liste ci-dessus.
4. Tester les pages, les langues, le panneau, la recherche et, avec les réservations activées, une réservation de test.
5. Dans le panneau, **désactiver le mode maintenance**.

Où se trouvent les sauvegardes

Quoi	Où et contenu	Comment restaurer
Sauvegardes automatiques du panneau	Sur le serveur, `data/backups/AAAA-MM-DD_HHMMSS/` : un dossier par enregistrement dans le panneau, les **60 derniers**. Tous les data/*.json (appartements, traductions, équipements, galeries, avis, table Beds24) et users.json, tels qu'ils étaient **avant** l'enregistrement.	Copier les fichiers dans data/ (users.json dans admin/) puis exécuter node scripts/sync-data.js pour régénérer les .js et les galeries (sans Node sur le serveur : sur une copie locale, puis envoyer les fichiers générés).
Code	GitHub — historique git.	git log ; git checkout <commit> -- <fichier>.
Photos	img/ sur le serveur. **Pas de sauvegarde automatique.** Supprimer une photo dans le panneau la retire seulement du site ; le fichier reste dans img/.	La resélectionner dans le panneau ; si le fichier est perdu, copie manuelle ou de l'hébergeur.
Réservations	Beds24 (source de vérité) ; data/reservations/ garde l'état de chaque paiement.	Consulter Beds24 et Stripe.
Configuration et comptes	api/config.php et admin/users.json, sur le serveur (jamais dans git).	En garder une copie hors du serveur, en lieu sûr.
Sauvegardes de l'hébergeur	Panneau de l'hébergeur (p. ex. cPanel → Backups).	Recommandé : sauvegardes quotidiennes de tout le site.
Archive locale	_arquivo/, uniquement sur l'ordinateur de développement : fichiers inutilisés par le site.	Voir _arquivo/LEIA-ME.md.

La même procédure et le même tableau figurent dans `DEPLOY.md` (en portugais), dans le dépôt.

Après la mise en ligne

- **Surveillance** de disponibilité (uptime) et alertes d'erreurs PHP.
- **E-mails :** SPF, DKIM et DMARC sur le domaine pour que les e-mails de réservation n'arrivent pas en spam.
- Contrôler régulièrement le refresh token Beds24 (il expire après 30 jours sans utilisation).

> **Prévisualisation locale :** tant qu'il n'y a pas d'hébergement, le site peut être consulté avec `php -S 127.0.0.1:8080 -t .` dans le dossier du projet.

6. SEO technique (avant la publicité)

Les annonces envoient du trafic vers des pages que Google évalue aussi (qualité de la page de destination). Ces éléments n'existent pas encore et devraient être faits avant de lancer des campagnes :

- **title** et **meta description** propres à chaque page (aucune meta description aujourd'hui).
- Balises **Open Graph** (image et texte lors du partage sur les réseaux sociaux / WhatsApp).
- **favicon** et icône pour mobile (inexistants).
- **robots.txt** (bloquer /admin/, /api/, /data/) et **sitemap.xml** avec les pages et les 60 appartements (`property.html?p=<slug>`).
- **Google Search Console** : valider le domaine et soumettre le sitemap.
- **Google Business Profile** de l'entreprise (adresse, téléphone, photos, avis).
- **Limite à connaître :** la page de chaque appartement est un modèle unique rempli en JavaScript, et le changement de langue se fait aussi en JavaScript sur la même URL. Google sait lire le JavaScript, mais EN et FR n'ont pas d'URL propres. Pour un SEO international solide, envisager des pages par langue ou un rendu côté serveur.
- Images : 17 photos de 3 appartements sont chargées directement depuis Airbnb (a0.muscache.com) — les télécharger dans img/.

7. Google Ads et mesure

Ce qui est déjà prêt

- Bandeau cookies accepter/refuser sur toutes les pages, qui enregistre le choix dans `localStorage["tmh_cookie_consent"]` et expose `window.TMHConsent` (`isAccepted()`, `onChange(fn)`, `open()`) ainsi que l'événement `tmh-cookie-consent`.
- Page de confirmation `reserva.html`, qui connaît l'état, la référence et le montant de la réservation — l'endroit naturel pour enregistrer la conversion.

Comptes à créer / relier

- Google Ads et Google Analytics 4 (reliés entre eux) ; en option Google Tag Manager.
- Google Search Console et Google Business Profile (voir section 6).

Consentement (Consent Mode v2)

Dans l'Espace économique européen, Google exige le Consent Mode v2 pour utiliser les données publicitaires. La balise Google doit démarrer avec tout refusé, puis être mise à jour par le bandeau cookies du site. Exemple, à placer dans le <head> de toutes les pages, avant le script gtag :

```
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){ dataLayer.push(arguments); }
  gtag('consent', 'default', { ad_storage: 'denied', ad_user_data: 'denied',
       ad_personalization: 'denied', analytics_storage: 'denied' });
</script>
<script async src="https://www.googletagmanager.com/gtag/js?id=AW-XXXXXXXXX"></script>
<script>
  gtag('js', new Date()); gtag('config', 'AW-XXXXXXXXX'); gtag('config', 'G-XXXXXXXXXX');
</script>
```

Puis, après js/cookie-consent.js, relier le choix du visiteur :

```
function tmhApplyConsent(accepted) {
  var v = accepted ? 'granted' : 'denied';
  gtag('consent', 'update', { ad_storage: v, ad_user_data: v,
       ad_personalization: v, analytics_storage: v });
}
tmhApplyConsent(TMHConsent.isAccepted());
TMHConsent.onChange(function (s) { tmhApplyConsent(s === 'accepted'); });
```

> Le texte du bandeau cookies ne parle que de « mesurer l'audience ». Avec de la publicité, il doit aussi mentionner les cookies publicitaires, et la politique de cookies doit lister ceux de Google.

Conversions

Dans `js/booking-result.js`, lorsque la réservation passe à confirmed, envoyer la conversion une seule fois (mémoriser la référence dans sessionStorage pour éviter les doublons au rechargement) :

```
gtag('event', 'conversion', {
  send_to: 'AW-XXXXXXXXX/YYYYYYYY',   // action de conversion « Réservation confirmée »
  value: r.total, currency: 'EUR', transaction_id: r.id
});
```

- Conversions secondaires utiles : demande de réservation sans paiement (statut request), clics sur le téléphone et l'e-mail, envoi du formulaire propriétaires.
- Tout valider avec Google Tag Assistant avant d'activer les campagnes.

Campagnes (proposition de départ)

- **Search — marque** (« TheMajord'Home ») et **générique** (« location appartement Porto Baixa », « appartement Ribeira »…), avec les pages des appartements comme destination.
- **Propriétaires :** campagne séparée « gestion location saisonnière Porto » → pages de services.
- Vérifier si le compte Beds24 permet une connexion à Google (Vacation Rentals / Hotel Center) pour apparaître dans les résultats d'hébergement de Google.
- N'activer les campagnes de réservation qu'une fois le paiement branché et testé.

8. Points en suspens et points d'attention

Les points juridiques doivent être validés par un juriste ou un expert-comptable ; cette liste indique quoi vérifier, elle ne remplace pas ce conseil. Le site est soumis au droit portugais.

Juridique (critique avant d'accepter des paiements et de la publicité)

Point	Situation
Mentions légales	legal-notice.html est un modèle français (lois françaises, CNIL) et cite un autre site (collenia.com). À refaire selon le droit portugais, en PT/EN/FR.
Politique de confidentialité	Inexistante. Nécessaire (RGPD) : les réservations collectent nom, e-mail, téléphone ; les données transitent par Beds24, Stripe et Google.
Politique de cookies	Inexistante. Nécessaire, surtout avec Google Analytics/Ads.
Conditions de réservation	Inexistantes : annulation et remboursement, acompte, arrivée/départ, règlement intérieur, responsabilité.
Enregistrement « alojamento local »	Vérifier l'obligation d'afficher le numéro d'enregistrement (RNAL) de chaque appartement dans l'annonce.
Livro de Reclamações	Vérifier l'obligation de lien vers le Livro de Reclamações électronique et l'information sur la résolution alternative des litiges.
Identification de l'entreprise	Raison sociale, NIF et adresse sur le site.

Point	Situation
Sous-traitants (RGPD)	Accords de traitement des données avec Beds24, Stripe, Google et l'hébergeur.
Droits sur les images	Les photos envoyées pour les services provenaient d'autres entreprises / banques d'images ; vérifier les droits. Photos et avis issus d'Airbnb : vérifier qu'ils peuvent être utilisés.

Contenu

- Les 4 pages de services ne sont qu'en portugais (EN/FR manquants).
- Les galeries des services utilisent provisoirement des photos d'appartements (sauf concierge-2.jpg).
- Aucun appartement n'a de prix « à partir de » : « Prix sur demande » s'affiche partout tant que Beds24 n'est pas branché.
- Les 60 appartements sont tous mis en avant (demande de la cliente) — l'accueil affiche les 60.
- Aucun appartement « Pretty Porto » n'existe (une demande de suppression le mentionnait).

Technique

- `js/reviews-data.js` démarre comme une liste vide alors que le code le traite comme un objet (corrigé par le panneau à l'approbation du premier avis).
- Les appartements ont un champ « price » inutilisé (peut être retiré des données).
- Pas d'analytics ni de tests automatisés.

9. Checklist de lancement

Avant la mise en ligne

☐ Textes juridiques refaits et publiés (mentions légales, confidentialité, cookies, conditions de réservation).
☐ Hébergement PHP 8.1+ et HTTPS ; domaine pointé.
☐ Dépôt publié sans .claude/, _arquivo/ ni fichiers isolés.
☐ api/config.php créé ; compte du panneau créé et mot de passe changé.
☐ Droits d'écriture PHP vérifiés ; display_errors désactivé.
☐ Meta descriptions, favicon, robots.txt, sitemap.xml.

Réservations

☐ Refresh token Beds24 configuré ; 60 roomId renseignés.
☐ Points VERIFICAR de api/beds24.php validés dans le Swagger.
☐ Stripe en mode test : réservation complète avec la carte 4242…, webhook reçu, réservation confirmée dans Beds24.
☐ Expiration testée (réservation annulée et dates libérées).
☐ E-mails envoyés par SMTP ; SPF/DKIM/DMARC configurés.
☐ Clés live ; interrupteurs en 'live' ; texte property.finePrint mis à jour.

Google

☐ Search Console validée et sitemap soumis ; Google Business Profile.
☐ GA4 + Google Ads avec Consent Mode v2 relié au bandeau cookies.
☐ Conversion « Réservation confirmée » validée avec Tag Assistant.
☐ Texte du bandeau et politique de cookies mentionnant la publicité.

Ensuite

- ☐ Sauvegardes quotidiennes de l'hébergeur actives ; api/config.php et admin/users.json conservés hors du serveur.
- ☐ Surveillance de disponibilité et des erreurs.
- ☐ Contrôler les premières réservations réelles dans Beds24 et Stripe.